

LabQuiz suite documentation

Wednesday 10th June, 2026



1 labQuiz suite documentation

1.1 LabQuiz overview

labquiz is a Python package that allows you to seamlessly integrate interactive quizzes directly into Jupyter notebooks — useful for labs, tutorials, practical assignments, continuous assessment, and controlled exams.

It combines:

- Multiple-choice and numerical questions
- Template-based parameterized questions
- Configurable number of attempts
- Hints and detailed feedback
- Automatic scoring
- Optional remote logging (Google Sheets)
- Real-time monitoring dashboard (if logging)
- Integrity checks and anti-tampering mechanisms

And it comes with two optional companion tools:

- **quiz_editor** — Create, edit, encrypt, and export question banks
- **quiz_dash** — Monitor, correct, and analyze results in real time

Together, these components make up the **LabQuiz** ecosystem, which is distributed as a bundle, **labquizbundle**.

Figure: Overview of the **LabQuiz** ecosystem. Dashed arrows indicate optional components. YAML quizzes drive both interactive notebooks and exports, monitored via a dashboard producing analytics, marks tables, and student reports.

- **Live version** Try it in [binder](#)
- **Installation:** All packages can be installed at once (**labquiz**, **quiz_editor** and **quiz_dash**)

From PyPI
pip install labquizbundle
or individually, e.g.

```
pip install labquiz
```

They can also be installed from source e.g.

```
# From source
```

```
pip install git+https://github.com/jfbercher/labquiz.git#subdirectory=quiz_editor
```

with subdirectories `meta` (the bundle) , `quiz_nb` (for labquiz), `quiz_editor` and `quiz_dash`.

1.1.1 Why LabQuiz?

LabQuiz is designed for **active learning** and **controlled assessment** in computational notebooks.

It helps instructors:

- Increase student engagement with embedded exercises
- Provide structured feedback during lab sessions
- Monitor progress in real time
- Run controlled tests and exams
- Detect configuration tampering or integrity violations

It helps students:

- Learn through interaction and immediate feedback
 - Track their progress
 - Work within structured assessment modes
-

1.1.2 What LabQuiz Does

Inside your notebook, you can:

- Add multiple-choice questions (`mcq`)
- Add numerical questions with tolerance (`numeric`)
- Create parameterized template questions
- Limit attempts

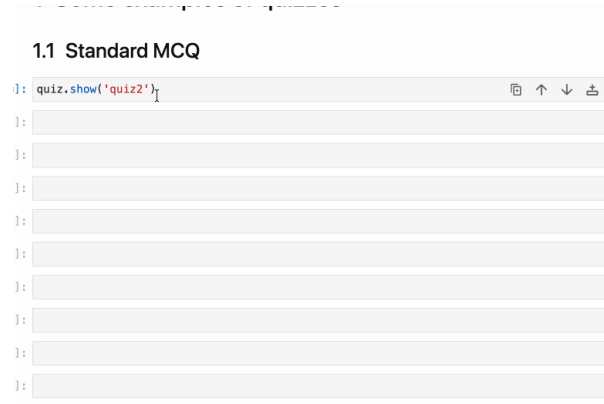
- Provide hints and corrections
- Compute automatic scores
- Log all activity to a Google Sheet backend (optional)
- Enable exam mode with integrity checks

Example:

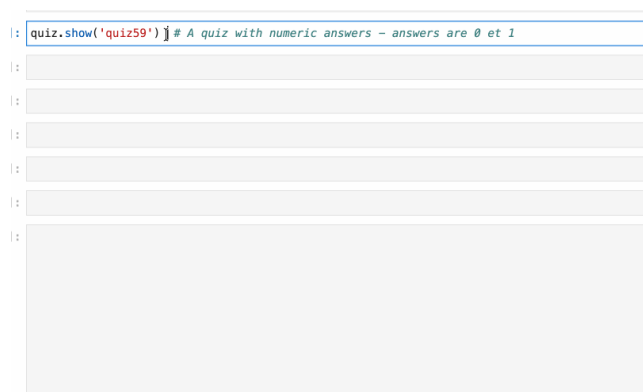
```
from labquiz import QuizLab

quiz = QuizLab(URL, "my_quiz.yml", retries=2, exam_mode=False)
quiz.show("quiz1")
```

1.1.3 Examples



Multiple-choice question (with hints & correction)



Numerical question

```
[ ]: tp = 22; tn = 21; fp = 8; fn = 7; |
quiz.show('quiz279', TP = tp, TN = tn, FP = fp, FN = fn)#, autovars=True)
```

Template-based question (dynamic variables)

```
[ ]: quiz.show('quiz81', autovars=True) |
```

```
[ ]: quiz.show('quiz82', autovars=True) |
```

```
[ ]: #####
```

1.1.4 Question Types, Pedagogical modes, Logging

Question types LabQuiz supports four types:

Type	Description
<code>mcq</code>	Standard multiple-choice
<code>numeric</code>	Numerical answers with tolerance
<code>mcq-template</code>	Context-dependent MCQ
<code>numeric-template</code>	Context-dependent numerical questions

Template questions allow dynamic evaluation based on runtime variables — ideal for practical lab computations.

Example:

```
quiz.show("quiz54", a=res1, b=res2)
```

Variables can also be generated dynamically

```
quiz.show("quiz54", autovars=True)
```

The expected solution is dynamically computed using Python expressions.

Pedagogical modes LabQuiz supports three pedagogical modes:

- Learning mode (hints + correction available, score display)
- Test mode (limited attempts, score display but no correction)
- Exam mode (no feedback, secure logging)

Quizzes are defined in simple YAML format and support

- Logical constraints (XOR, IMPLY, SAME, IMPLYFALSE)
- Bonuses and penalties
- Relative and absolute tolerances
- Variable generation for templates

Remote Logging & Dashboard All data can be stored in a **Google Sheet backend**.

LabQuiz can log: Validation events, Parameters, User answers, Integrity hashes... LabQuiz also includes multiple anti-cheating mechanisms (Machine fingerprinting, Source hash verification, Detection of parameter tampering, Optional encrypted question files, Runtime integrity daemon...)

1.1.5 Installation

```
pip install labquiz
```

```
pip install git+https://github.com/jfbercher/labquiz.git
```

Import:

```
import labquiz
from labquiz import QuizLab
```

Instantiate:

```
quiz = QuizLab(URL, QUIZFILE,
               retries=2,
               needAuthentication=True,
               mandatoryInternet=False)
```

1.1.6 Additional Tools

quiz_editor — Build & Export Question Banks Creating YAML files manually is possible — but **quiz_editor is intended to make it easier.** It can also be useful outside of LabQuiz as a general quiz-editor with export capabilities and large markdown support (including images, tables, equations, etc).

Key features:

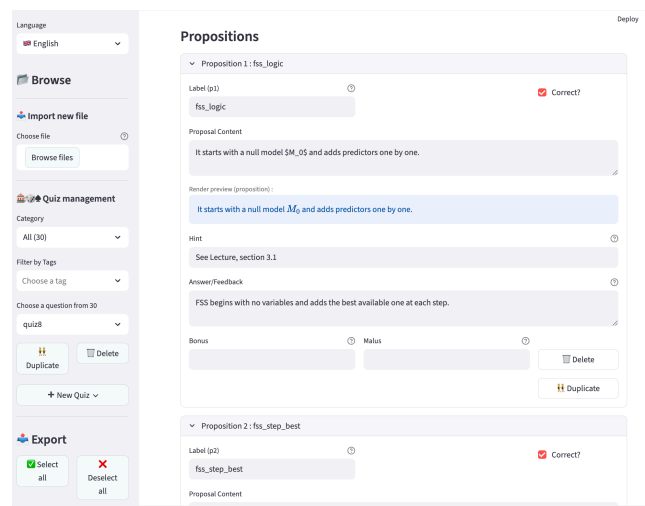
- Visual question editing (MCQ, numeric, templates)
- Categories & tags
- Variable generation for templates
- Bonus / malus configuration
- Logical constraints (XOR, IMPLY, SAME, etc.)
- One-click export to:
 - YAML
 - Encrypted version

- Interactive HTML (training mode), with dynamic support for re-generating variable values in template questions
- HTML exam version (Google Sheet connected)
- AMC–LaTeX format (paper exams), static and dynamic versions (using pythonTeX)

Online version: <https://jfb-quizeditor.streamlit.app/>

Install locally:

```
pip install quiz --editor
```



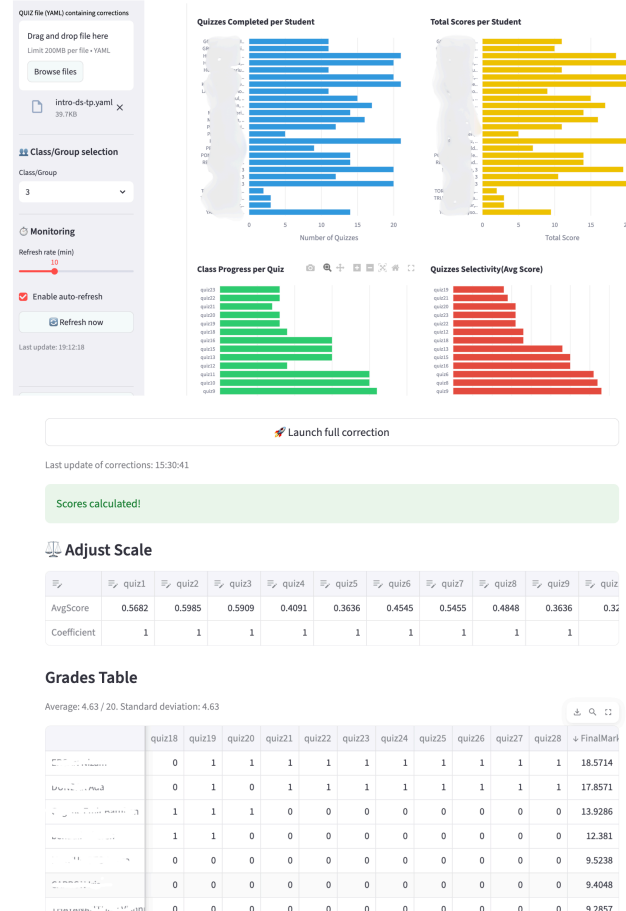
quiz_dash — Real-Time Monitoring & Correction quiz_dash is the companion dashboard for instructors.

It connects to your Google Sheet backend and provides:

- Live tracking of submissions
- Live class overview
- Student-by-student monitoring
- Integrity checks (mode changes, retries tampering, hash verification)
- Adjustable grading weights and scale
- Automatic recalculation
- Full individual corrections available for download

- CSV export of results

Online version: <https://jfb-quizdash.streamlit.app/>



Optional: Zero Installation with JupyterLite LabQuiz can run entirely in the browser using JupyterLite (WASM). Perfect for fully web-based lab environments.

1.1.7 Ecosystem

Tool	Purpose
labquiz	Notebook quiz engine
quiz_editor	Question bank creation & export
quiz_dash	Monitoring & correction dashboard

Repositories:

<https://github.com/jfbercher/labquiz> is a multipackage repository, that includes the [bundle](#), and

- the [labquiz](#) notebook package,
- the [quiz_editor](#) package,
- the [quiz_editor](#) package

Online tools:

- <https://jfb-quizeditor.streamlit.app/>
 - <https://jfb-quizedash.streamlit.app/>
-

1.1.8 Typical Workflow

1. Prepare questions (YAML or `quiz_editor`)
 2. Optionally encrypt file
 3. Create Google Sheet backend
 4. Instantiate `QuizLab` in notebook
 5. Run lab / test / exam
 6. Monitor using a python console or with `quiz_dash`
 7. Post-correct with adjustable grading
-

1.1.9 Demonstration

See:

- `labQuizDemo_en.ipynb` in `extras/`
 - Live version Try it in [binder](#)
-

1.1.10 License

GPL-3.0 license

1.2 Introduction & first examples

LabQuiz in a nutshell

It is a Python package that

- allows you to integrate questions in the form of multiple-choice questions or numerical values,
- with multiple attempts (adjustable), hints, and *feedback*,
- correct quizzes online or *a posteriori*,
- automatically authenticates and records responses on a remote server,
- provides a dashboard to track student progress,
- and more

Learning objectives

- encourage student engagement through exercises and *fun* assessment,
- monitor and measure the results to be achieved during the session,
- allow students to *take a step back* by linking the results to the course and going into more depth,
- encourage learning through progress indicators and *feedback*,
- monitor, using using a *dashboard*, monitor student progress over time,
- as well as the entire group,

Links

Source code

- https://github.com/jfbercher/labquiz/tree/main/quiz_nb
- https://github.com/jfbercher/labquiz/tree/main/quiz_editor
- https://github.com/jfbercher/labquiz/tree/main/quiz_dash

Online

- <https://jfb-quizeditor.streamlit.app/>

- <https://jfb-quizdash.streamlit.app/>

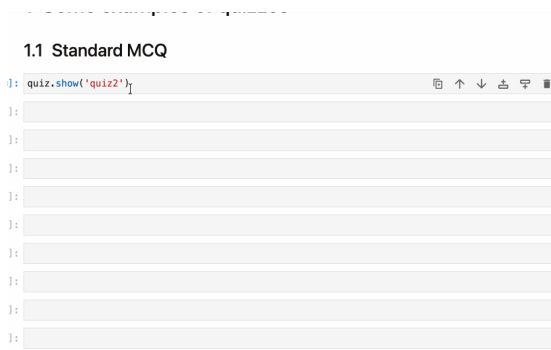
Table of contents

To display a table of contents, press the table of contents button on the left; or menu **View/Table of Contents**

1.2.1 First examples

Quizzes are integrated into the subject throughout the tutorial or practical exercise, based on a questionnaire prepared in advance.

A first example to illustrate a multiple-choice quiz, with tips and corrections:



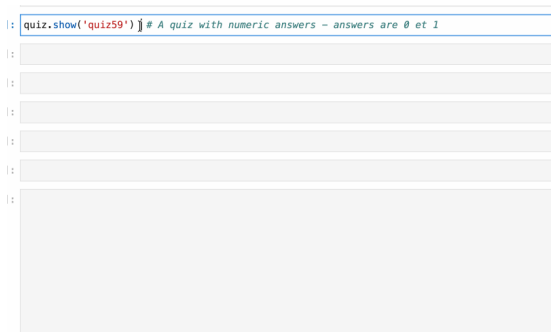
1.1 Standard MCQ

```
1: quiz.show('quiz2')
```

10 empty input fields for multiple-choice answers.

Figure 1: Question during the lab session

Second example with numerical values



```
1: quiz.show('quiz59') # A quiz with numeric answers - answers are 0 et 1
```

5 empty input fields for numerical answers.

Figure 2: Another question during the practical work

(with the quiz blocked if the answer has already been submitted!)

Following examples present **template-based questions** (use of values passed from notebook's context, or dynamically generated)

```
[ ]: tp = 22; tn = 21; fp = 8; fn = 7  
quiz.show('quiz79', TP = tp, TN = tn, FP = fp, FN = fn)#, autovars=True)
```

Figure 3: Example of confusion matrix with dynamic counts (and expected values)

```
[ ]: quiz.show('quiz81', autovars=True)
```

Figure 4: Example of linear regression - computation of prediction with dynamic variables

Connected mode

In connected mode, i.e. as soon as a valid URL has been passed during initialization

- all entries (Validate, Correct) are recorded and transmitted,
- a specific identifier for each machine is calculated and used to identify all logs,

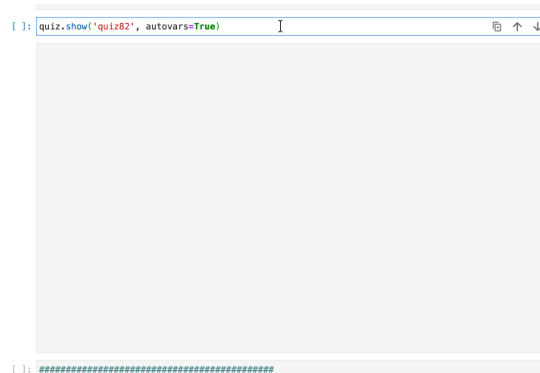


Figure 5: MCQ for a linear regression example

- a system status is transmitted periodically, with a known and verifiable period, allowing the detection of
- changes to settings (exam mode, number of attempts allowed, etc.),
- changes to the package source,
- code injection,
 - a kernel restart, a new instantiation, a machine change...
- etc Of course, any interruption in periodic transmission by a duly registered machine is necessarily detected...

In the following

- the label refers to information specific to teachers or subject designers
- the label refers to information specific to students

1.3 Installation & Usage

1.3.1 Installation

All packages can be installed at once (`labquiz`, `quiz_editor` and `quiz_dash`)

From PyPI

```
pip install labquizbundle
```

or individually, e.g.

```
pip install labquiz
```

They can also be installed from source e.g.

From source

```
pip install git+https://github.com/jfbercher/labquiz.git#subdirectory=quiz_editor
```

with subdirectories `meta` (the bundle), `quiz_nb` (for `labquiz`), `quiz_editor` and `quiz_dash`. Use `pip` in a terminal, or `%pip ...` in a notebook cell.

For `quiz_editor` and `quiz_dash`, entry points are provided, allowing to use the apps directly from the command line by simply invoking

```
quiz_dash
```

or

```
quiz_editor [FILENAME.yaml]
```

It is possible to use a standalone version, *without any Python installation*, which runs in the browser. This is described in Section [1.11.1](#).

1.3.2 Usage

Once `labquiz` is installed, import it with

```
import labquiz
```

```
from labquiz import QuizLab
```

and instantiate a quiz with

```
URL = "" # path to a URL to collect results
```

```
QUIZFILE = "name_of_quiz_file.yaml" # for example "quizzes_basic_filtering_test.yaml"
```

```
quiz = QuizLab(URL, QUIZFILE)
```

Additional parameters can be specified (default values below)

```
needAuthentication=True,      # Authentication required yes/no
retries=2,                    # number of possible attempts = retries + 1
exam_mode=False,              # default False
test_mode=False,              # default False
groups=[],                    # list of groups (or classes) of students, eg ['A', 'B', ...]
mandatoryInternet=False,      # requirement to have a valid internet connection (if False, it will try to connect)
CHECKALIVE=60,                 # integrity check every CHECKALIVE seconds
INACTIVITY_TIMEOUT=3600,       # suspend the integrity daemon if no activity record for INACTIVITY_TIMEOUT seconds
in_streamlit=False,           # put it to True is quiz is instancied in streamlit
#(no changes to the program or parameters)
```

```

1: import labquiz
2: from labquiz import QuizLab
3: labquiz.set_language('en')
4:
5: #URL without sheetID | Here we are in stand-alone mode, no internet connection required, re:
6: URL = ""
7: QUIZFILE = "test.yaml"
8: quiz = QuizLab(URL, QUIZFILE, needAuthentication=True, retries=99)
9:
10:
11:
12:
13:

```

Figure 6: Login example

1.3.3 Features

LabQuiz also works in **visual studio code**, but $\LaTeX$ portions are not rendered in questions, suggestions, and answers. This is a limitation of Visual Studio, which does not load MathJax, and may be improved in the future. In the meantime, it is better to use a classic Jupyter, Colab, or JupyterLite. In Visual Studio, this can be managed, but it is less effective if there is $\LaTeX$ in the text.

Types of quiz questions Four types of questions are available:

- multiple-choice questions (**type:** "qcm" in the question file; this is the default type)
- numerical questions (**type:** "numeric")
- Context-dependent multiple-choice questions (**type:** "qcm-template")
- Context-dependent numerical questions (**type:** "numeric-template") The structure of the question file, where these different types are indicated, is presented Section 1.4.1. How to prepare and even encrypt this file is presented in .

Templates allow you to ask questions that depend on local variables. This allows you to test specific values, orders of magnitude, the result of a calculation, or the consistency between several values. Here are two examples to illustrate this:

Different presentation modes

Learning mode In learning mode, all four buttons are available. The validate button displays the score obtained, see Figure 8. The tips button displays advice. The correct button displays the correction, see Figure 9 and Figure 10. The boxes turn green if they have been checked or unchecked correctly, and red if not. The checks entered by the user are retained, but colored green or red depending on the correct result. After the correction has been requested, the validate, reset, and tips buttons are disabled and become inoperative.

1.3 Values passed as parameters; type "mcq-template"

```
1: # A test with values passed as parameters; type "mcq-template"
a = 2
b = 7
quiz.show('quiz53', A=a, B=b)

2: # A test with values passed as parameters; type "mcq-template"
# autovars=True --> Generate values on the fly
quiz.show('quiz53', autovars=True)
```

1.4 Test mode

```
1: # A test with values passed as parameters; type "numeric-template"
# and switching to test mode (no correction)
quiz.set_test_mode()
a = 2
b = 11
quiz.show('quiz54', a=a, b=b)
```

Figure 7: Template questions that use numerical values passed as parameters

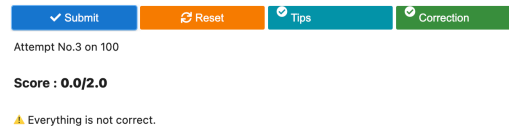


Figure 8: Learning mode - the submit button has been pressed



Figure 9: Learning mode - the corrected button has been pressed

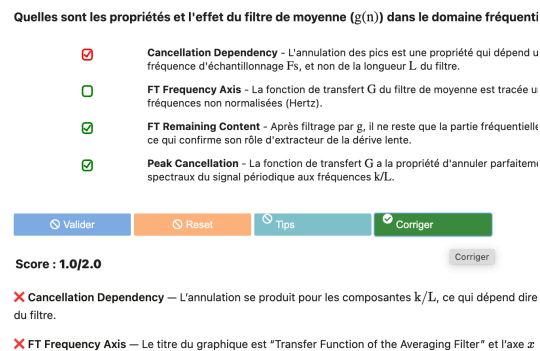


Figure 10: Learning mode - the correct button has been pressed and the correction is displayed

Test mode In test mode, the correct button is removed. The student sees their score after validation and tips are possible. The number of validations is limited by the `retries` parameter passed at initialization.



Figure 11: Test mode - the validate button has been pressed

Exam mode In exam mode, there is no score display, no tips, and no corrections. Only the reset and validate buttons appear.



Figure 12: Exam mode - the submit button has been pressed

Individual questions or series of questions `exam_show` Regardless of the presentation modes described above, questions can be presented either individually with a call of the form `quiz.show('label')`, or in a block of questions. The latter option can be useful for a short interim review, for example.

A block of questions will be presented using the `quiz.exam_show()` function with the following parameters:

```
exam_show(exam_title="", questions=None, shuffle=False, nb=None)
```

- `exam_title`: Used to identify the exam, default ""
- `questions`: List of question labels (e.g. "quiz1," "quiz2"). If None, uses all questions in the bank.
- `shuffle`: default False; If True, shuffles the order of questions before display.
- `nb`: If not None, randomly draws nb distinct questions from the set of available questions.

An example of a call could be

```
q1 = quiz.exam_show(exam_title="Test to see", shuffle=True, nb=4)
```

Intermediate results are not displayed (if in learning or test mode). The results obtained can then be viewed by the student with `quiz.exam_result(q1, marking_scheme=None)`.

In exam mode, results are not calculated or viewable, and the teacher can correct all exams as described in .

1.4 Question file structure

1.4.1 Question file structure -

General structure The question file is a [YAML](#) file. The format is simple, but the number of spaces or indentations must be consistent throughout the file.

The general structure is as follows:

QuizFile

```
title (string) [optional]

quiz_id (ex: quiz1, quiz23, ...)

  question (string)
  type (mcq | numeric | mcq -template | numeric -template) [optional]
  label (string)

  constraints [optional]
    list
      constraint
        indexes [list of labels]
        type (XOR | SAME | IMPLY | IMPLYFALSE)
        malus [optional]

  variables [templates only]
    variable_name
      type
      structure
      engine
      call

  propositions (list)

    proposition
      proposition (string)
      label [optional]
      type (bool | float | int | numeric) [optional]
      expected (any)
      tolerance [numeric only]
      tolerance_abs [numeric only]
      tip [optional]
      answer [optional]
      bonus [optional]
      malus [optional]
```

with the following relationships

The file begins with a line

```
title: an explanation of the file 's contents
```

It then contains a list of quizzes, each beginning with a label, for example (but there are no restrictions on the choice of labels)

```
quiz1:
...
quiz2:
...
```

Each quiz itself contains a question, followed by a series of options.

- There is no limit to the number of options; there must be at least one.
- The texts of the questions and answers are character strings, which can be enclosed in single or double quotation marks. These quotation marks are not mandatory, unless the string contains a :, in which case you must enclose it in single quotation marks (and in this case, if the string also contains single quotation marks, they must be doubled, see the examples below)

```
quiz23:
  question: This is the text of the question
  ...
  propositions:
    - proposition: text of the first proposition
    ...
    - proposition: 'text of the second proposition with a : that must be taken
    ...
```

This is the bare minimum. By default, the quiz is of the “mcq” type. It can also be of the “numeric” type, in which case this must be specified. If you want to be able to use online correction and display tips, you will need to add them.

```
quiz23:
  question: This is the text of the question
  type: "mcq"          #"mcq" or "numeric" (optional - "mcq" by default)
  propositions:
    - proposition: text of the first option (incorrect)
      type: bool        # optional - default "bool" if type "mcq", "float if t
      label: label1     # optional but necessary for corrections
      expected: false   # expected value for the answer
      tip: text of a clue or hint towards the correct answer
      answer: explanatory text for the correct answer
    - proposition: 'text of the second proposition (true) with a : that must
      label: label2
```

```

        type: bool
        expected: true
        tip: text of a clue or guidance towards the correct answer, with quotes
        answer: explanatory text for the correct answer, with quotes '
if necessary

```

- *Consistency constraints* on propositions can be added. For example, it can be required that the true answer to the proposition of label `label12` implies that the answer to the proposition `label11` is false. In case of violation, a penalty is applied.
- Similarly, certain propositions can give rise to a *bonus* or a *penalty*. The bonus is the number of points awarded if the answer is the expected one (default 1) and the penalty is the number of points deducted if the answer is different from the expected one (default 0). With these elements, the example could be completed as shown below. The implementation is then given Figure 13.

```

quiz23:
  question: This is the text of the question
  type: "mcq"      #"mcq" or "numeric" (optional — default "mcq")
  constraints: [
    { "indexes": ["label2", "label1"], "type": "IMPLYFALSE", "penalty": 2 }
  ]
  propositions:
    — proposition: text of the first proposition (false)
      type: bool          # optional — default "bool" if type "qcm", "float if t
      label: label1       # optional but necessary for corrections
      expected: false     # expected value for the answer
      tip: text of a clue or hint towards the correct answer
      answer: explanatory text for the correct answer
    — proposition: 'text of the second proposition (true) with a : that must be
      label: label2
      type: bool
      expected: true
      tip: text of a clue or guidance towards the correct answer, with quotes
      answer: explanatory text for the correct answer, with quotes ' if neces
    — proposition: text of a third proposition with penalty
      label: label3
      type: bool
      expected: true
      malus: 2           #penalty applied here if the box is not checked
      tip: text of a hint

```

Several logical constraints can be specified, as in this example:

```

1: quiz.set_normal_mode()
2: quiz.show('quiz23')

```

This is the question text

☐ text of the second proposition (true) with a ; that must be taken into account
☐ text of a third proposition with penalty
☐ text of the first proposition (false)

```

1: 
2: 
3: 
4: 

```

Figure 13: Implementation of question `quiz23`. Note that the propositions are automatically mixed.

```

quiz57:
  question: "This is a question with contradictions and implications. The number
  type: "mcq"
  constraints: [
    { "indexes": ["parity", "odd"], "type": "XOR", "penalty": 2 },
    { "indexes": ["parity", "multiple of 2"], "type": "SAME", "penalty": 2 },
    { "indexes": ["parity", "plus1pair"], "type": "XOR", "penalty": 2 },
    { "indexes": ["parity", "plus1odd"], "type": "IMPLY", "penalty": 2 }
  ]

```

Type numeric For questions with numerical values, the pattern is similar. Additional keys are available: `tolerance`, `tolerance_abs`

- `tolerance` is the percentage of variation tolerated on the expected value
- `tolerance_abs` is the absolute tolerance.

The tolerance used during correction is the greater of the values between `tolerance_abs` and `tolerance*expected`. The `type` in each proposition can be `float` (default) or `int`. Bonuses (default 1) and penalties (default 0) can also be specified and are applied depending on whether the difference between the given value and the expected value is greater or less than the tolerance.

```

quiz24:
  question: Please enter below the number of points and the values of the mean a
  type: numeric
  propositions:
    - proposition: Mean:
      label: mean
      type: float

```

```

expected: 0.0
answer: 0
tolerance: 0.05
tolerance_abs: 0.01
tip: Enter the value
- proposition: Standard deviation
label: sigma
type: float
expected: 1.0
answer: "1"
tolerance: 0.05
tolerance_abs: 0.01
tip: Enter the value
- proposition: Number of points
label: N
type: int
expected: 512
answer: The number of points 'len(series)' or 'series.shape[0]' is 512
tolerance: 0.01
tolerance_abs: 2
bonus: 2
penalty: 3
tip: Enter the value

```

Figure 14: Implementation of the `quiz24` question of the numeric type. Note that the propositions are automatically mixed and that a bonus/penalty is applied to the number of points.

Case of templates Two additional types are possible, namely `numeric-template` and `qcm-template`. These formats allow the use of variable numerical data and values related to the experiments carried out in the practical. This data is passed as a parameter to the `show()` function and used for correction. For

multiple-choice questions, it is possible to test whether the result belongs to an interval or other calculable conditions whose result is Boolean. For correction, a formula is implemented that calculates the expected value based on the parameters passed.

In the following example, the `show` function is called with two parameters:

```
quiz.show("quiz54", a=res1, b=res2)
```

These two parameters are used to calculate the solution. For example, the formula `f'{a+b:.4f}'` (invisible to the student!) is *evaluated* with the context that is passed to the function, and this context is simultaneously *saved* on the remote server to allow the teacher to recalculate the solution offline. The formula can also be specified naturally as “a+b” or as “{a + b}”, with or without quote or brace.

A field `variables` can also be precised in the quiz file, as

```
variables:
  a:
    type: int
    structure: scalar
    engine: numpy rng.
    call: integers(0, 10, size=1)
  b:
    type: int
    structure: scalar
    engine: numpy rng.
    call: integers(0, 10, size=1)
```

This enables to provide values for the different variables, which is useful for export to HTML or AMC-~~L~~A_TE_X format, but this also allows values to be generated on the fly if they are not given in the quiz. The `autovars=True` option is provided for this purpose, e.g.

```
quiz.show("quiz54", autovars=True)
```

```
quiz54:
  question: "This is a numerical question where you have to calculate the sum an
  type: "numeric -template"
  variables:
    a:
      type: int
      structure: scalar
      engine: numpy rng.
      call: integers(0, 10, size=1)
    b:
      type: int
      structure: scalar
      engine: numpy rng.
```



```

    call: integers(0, 10, size=1)
propositions:
- proposition: "Sum: "
  label: 'sum'
  type: "float"
  expected: f'{a+b:.4 f}'
  answer: The sum of {a} and {b} is {a+b}
  tolerance: 0.01
  tolerance_abs: 0.01
  tip: "Enter the value"
- proposition: "Difference: "
  label: "difference"
  type: "float"
  expected: f'{a -b:.4 f}'
  answer: The difference is {a -b}
  tolerance: 0.01
  tolerance_abs: 0.01
  tip: "Enter the value"

```

```

: res1 = 2.11
: res2 = 7.42
: quiz.show("quiz54", a=res1, b=res2)

```

Ceci est une question numérique où il faut faire la somme et la différence de 2.11 et 7.42

Somme :	9.53	Tolérance : max de ±1.0% ou ±0.01
Différence :	-5.31	Tolérance : max de ±1.0% ou ±0.01

Score : 0.0/2.0

Figure 15: Implementation of the `quiz54` question of numeric type. Parameters are passed to the function, which evaluates and calculates the correct solution based on these parameters.

Almost any Python function can be used to evaluate the answer.

Just keep in mind that the context is saved in the remote spreadsheet. Therefore, avoid making it too large: avoid large data tables! and prefer smaller contexts. Not all contexts and data types are necessarily serializable (dictionaries, lists, numpy arrays, and pandas are supported here). In addition, to recalculate the solution offline, you must store the useful modules.

In the following example, we calculate the coefficient of variation of a series using numpy. The call would be, for example

```
quiz.show("quiz61", s=np.random.randn(5), np=np)
```

where we pass the name of the module(s) used in the context.

`quiz61`:

```

question: "This is a numerical question where you have to calculate the coeffi
type: "numeric -template"
propositions:
  - proposition: "Coefficient of variation"
    label: "cv"
    type: "float"
    expected: np.std(s)/np.mean(s)

```

`s = np.random.normal(size=5)`
`quiz.show('quiz61', s=s)`

This is a numeric question where the coefficient of variation of the series `s=[-0.45395024 -1.80220557 -0.12016571 -1.33715716 -0.7442404]` must be calculated

Coef of var... Tolerance: maximum of ±1.0% or ±0.01

Submit Reset Tip Correction

Score : 0.0/1.0

The coefficient of variation is -0.6797025500850108

Figure 16: Implementation of the `quiz61` numeric question. Parameters are passed to the function, including a module that evaluates and calculates the correct solution based on these parameters.

1.5 Score calculation and correction by the teacher

1.5.1 Score calculation

A score is calculated automatically for each quiz and displayed in learning and test modes.

An overall score, which is the average of the results on the quizzes taken, can be accessed via `quiz.score_global`. The score is calculated using the two functions `calculate_quiz_score` and `correct_ans`, which are available in the `utils` and `putils` modules, respectively. The main principles are as follows:

- The function receives the user's answer `user`, a weight matrix, and the constraints imposed
- **weight matrix** - the *weight matrix* is used to weight the answers given according to the expected answer: number of points awarded if the user answered true when the expected answer was true, if they checked true when the expected answer was false, etc. The default matrix is as follows:

```
default_weights = {
    (True, True): 1, # True Positive # Checked and the correct a
    (True, False): -1, # False Positive # Checked when the correct
    (False, True): 0, # False Negative # Did not check though the
    (False, False): 0 # True Negative # Did not check and the cor
}
```

In short, here we award one point for correct answers that were checked and deduct one point for correct answers that were not checked. The weight matrix can be specified using the quiz `set_weight()` method, eg `quiz.set_weight(D)`, where `D` is a dictionary, with the default above. Depending on the case, we might want to adapt this to penalize correct answers that were not checked, or give less weight to an omission, for example

```
weights = {
    (True, True): 1, # True Positive # Checked and the correc
    (True, False): -0.5, # False Positive # Checked when the corr
    (False, True): -0.5, # False Negative # Not checked when the
    (False, False): 0 # True Negative # Not checked and the co
}
```

We can also impose an *identity* weight, which means that we award 1 point for each true statement checked and 1 point for each false statement not checked. A disadvantage of this strategy is that, obviously, a blank copy with no checks gets statistically half the points, if there are as many wrong statements as right ones, or even more if it is a “single correct answer” questionnaire.

```
weights = {
    (True, True): 1, # True Positive # Checked and the correc
    (True, False): 0, # False Positive # Checked when the correct
```

```

        (False, True): 0, # False Negative    # Did not check when the c
        (False, False): 1    # True Negative    # Did not check and the
    }

```

The weight matrix cannot be modified when calculating the score online, but it can be modified when recalculating retrospectively based on the recorded results; see correction by the teacher .

Inside a given question, all propositions have the same weight by default. However it is possible to adjust this using a system of bonus-malus. A proposition can be given a weight (bonus) of 2 while the other have a weight of one for example. Or a bad answer a malus of -2 while the default is -1. For a given question, the total score is always normalized to 1.

- **bonus malus** - As we have seen in the file structure Section 1.4.1, **bonuses** and **penalties** can be integrated into the questions themselves. This allows, for a given question, to give more weight to a particular correct answer, or conversely to penalize a wrong answer, independently of the general weighting matrix.
- **logical constraints** - *Logical constraints* can be integrated into questions and used to calculate the score. These constraints are specified question by question in the question file, and penalties are applied if the constraint is violated. The following constraints can be used:

```

# constraints: List of dictionaries e.g.: [{"indexes" : (0, 1), "type": "XOR, "p
    - XOR (Exclusion) A and B must be different
    - IMPLY (Implication) If A is TRUE, then B is TRUE
    - SAME (Consistency) A and B are equivalent (same value)
    - IMPLYFALSE: If A is TRUE, then B MUST be FALSE

```

See an example at the end of Section 1.4.1.

- **numerical values** - In the case of numerical answers, the difference between the given value and the expected value is calculated. If this difference is less than the threshold defined by the tolerance, the answer is counted as correct¹. Remember that the tolerance is specified in the question file, and that the greater of the values between `tolerance_abs` and `tolerance_relative*expected` is used. If the relative tolerance **tolerance** has not been specified, the value used is 1%. Bonuses (default 1) and penalties (default 0) may also be applied depending on whether the difference between the given value and the expected value is greater or less than the tolerance.

¹It would be possible to set the score based on the (relative) value of this difference, but this has not been done and is left for "later".

1.5.2 Correction by the teacher

In parallel with the practical work, or *a posteriori*, the teacher can use the automatically recorded results to monitor progress, adjust the weights (weight matrix) or the grading scale. Obviously, this is essential in **exam mode**, where students do not receive feedback on their answers and where the corrections are not accessible. To correct all recorded answers, the correction is as simple as:

```
from labquiz.putils import correctQuizzes
URL = "https://_URL_USED_TO_COLLECT_RESULTS"
SECRET = "SECRET_PASSWORD_SPECIFIED_IN_THE_SHEET"
QUIZFILE = "NAME_OF_QUESTION_FILE.yml" #quiz file CONTAINING the expected values
#
Res = correctQuizzes(URL, SECRET, QUIZFILE)
```

This provides a table of results in the following form

Res

1 ResDataFill	quiz1	quiz2	quiz3	quiz4	quiz5	quiz6	quiz7	quiz8	quiz9	quiz10	quiz11	quiz12	quiz13	quiz14	quiz15	maxpts	Note
	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	14.0	0.000000
	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	14.0	0.000000
	1.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	14.0	2.857143
	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	14.0	0.000000
	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	14.0	1.428571
	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	0.0	1.0	0.0	1.0	0.0	1.0	1.0	14.0	17.142857
Bercher Jean-François	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	14.0	2.857143
	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	0.0	1.0	1.0	1.0	0.0	0.0	0.0	14.0	15.714286
	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0	14.0	17.142857
	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	14.0	12.857143
	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	14.0	2.857143

Figure 17: Example of a results table (Students names were masked).

which can of course be exported, for example with

```
Res.to_csv("Results.csv")
```

1.5.3 Correction for an exam generated with quiz.exam_show()

For an exam generated with `quiz.exam_show()`, which is identified by the title given to it when it was created, the teacher can specifically correct the data collected on the server, according to (after retrieving the data with `readData` and instantiating a quiz – steps 1 and 3 above),

```
from labquiz.putils import getExamQuestions, getAllStudentsAnswers, correctAll
exam_questions = getExamQuestions("Test_to_see", data)
students = exam_questions.keys()
students_answers = getAllStudentsAnswers(students, data, maxtries=3)
correctAll(students_answers, quiz, data_filt, threshold=0,
           exam_questions=exam_questions, weights=None, marking_scheme=None, ma
```

```

from labquiz.putils import correctQuizzes
URL = "https://URL_USED_TO_COLLECT_AND_STORE_RESULTS"
SECRET = "SECRET_PASSWORD_SPECIFIED_IN_THE_SHEET"
QUIZFILE = "NAME_OF_QUESTION_FILE.yml" #quiz file CONTAINING the expected value
Res = correctQuizzes(URL, SECRET, QUIZFILE, title='Test 'title)

```

where the title of the test to be corrected is specified by the `title` parameter.

	quiz55	quiz56	quiz57	quiz4	quiz6	quiz7	quiz8	quiz9	quiz12	quiz14	quiz15	maxpts	Note
Legrand John	0.0	0.0	1.0	0.5	1.0	0.0	0.0	0.0	0.0	1.0	0.0	4.0	17.5
De La Boiserie Du Bout Du Parc Joséphine	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	1.0	4.0	20.0
*****	0.0	1.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	4.0	10.0
Nguyen Mai	0.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	0.0	1.0	4.0	20.0

Figure 18: Example of a results table (`exam_show`).

1.5.4 Correction options

Recall that a weighting matrix can be used to assign weights to the four possible response/expected outcome situations encountered in a multiple-choice questionnaire, as shown in . In addition, the different options within a given question can be weighted using a bonus–penalty system. Finally, the overall balance of the questionnaire is adjusted through a global scoring scheme—referred to as “coefficients”—which allows greater weight to be assigned to specific questions, each question initially having a maximum score of 1.

Additional options can be used during correction.

```
def correctQuizzes(URL, SECRET, QUIZFILE, title=None, threshold=0, weights=None,
```

- `title`: if `title` is not `None`, this indicates that it is the correction of a test with randomly selected questions of type `exam_show`, and whose title is `title`,
- `threshold`: `threshold=0` sets the scores for each question to zero (otherwise negative scores are possible) ; this is the default value, but if you want to allow negative scores per question, you can set it to -10, for example.
- `weights`: the weight matrix (dictionary) already discussed in Section 1.5.1
- `marking_scheme` (scale): weight of the different questions in the quiz. If there is no scale, all questions have the same weight for the calculation of the score. If the weight of a question is not specified, it defaults to 1. Example: `marking_scheme = {'quiz3':4, 'quiz55':0}` assigns a coefficient of 4 to question `quiz3` and neutralizes question `quiz55` (all other questions will have a weight of 1),
- `maxtries`: Number of attempts allowed. Correction is performed on the last attempt less than or equal to `maxtries` (and before any correction request, “Correct” button, if available).

1.5.5 Dashboard

During the session, you can retrieve the data table (see previous section) and view the results. The following basic functions allow you to monitor overall progress

```
# Reading data
URL = "https://URL_UTILISÉE_POUR_RECUEILLIR_LES_RÉSULTATS"
SECRET = "SECRET_PASSWORD_SPECIFIED_IN_THE_SHEET"
data, data_filt = readData(URL, SECRET)
df_last = data_filt.drop_duplicates(
    subset=["student", "quiz_title"],
    keep="last")
#The number of quizzes completed per student
quiz_count_by_student = (
    df_last.groupby("student")["quiz_title"]
        .apply(len)
        .sort_index(ascending=False)
)
# Scores per student
score_by_student = (
    df_last.groupby("student")["score"]
        .apply(np.sum)
        .sort_index(ascending=False)
)
# For the entire class
class_counts = (
    df_last["quiz_title"]
        .value_counts()
        .sort_index (
            key=lambda idx: idx.str.extract(r"(\d+)").astype(int)[0]
        )
    # alphabetical order
)
#
print("Number of quizzes completed\n\n", quiz_count_by_student)
print("Scores obtained\n\n", score_by_student)
print("Complete class\n\n", class_counts)
```

We can even put this in the form of a small graphical *dashboard* that we will refresh regularly. The dashboard mentioned above was finally formatted into a dedicated application, which is described in [Section 1.9.1](#).

1.6 Security

1.6.1 Principles

Various measures have been taken to ensure a reasonable level of contributor identification, compliance with instructions, and fraud prevention. These measures include

- Identification of the machine (via its software and hardware system), which provides an identifier used in all transactions.
- The quiz settings and status are retained if the quiz is re-instantiated. Only a kernel restart can reset everything (and in this case, all local data is also lost).
- Encryption of the question file: this is encrypted using a key calculated at runtime and possibly dependent on a key available on a remote server, parameter `mandatoryInternet=True`.
- In connected mode, which is mandatory if the `mandatoryInternet` parameter has been set, *all* validations and correction requests are recorded and transmitted, along with the answers given to the current quiz, which also allows for post-correction.
- A hash of the sources and parameters is transmitted, allowing for the detection of modifications to the sources – including monkey patching, or monitored parameters.
- A *daemon* also periodically transmits the system status, including, of course, the machine's identification. - - That being said, to conduct an *exam* under good conditions, use a question file WITHOUT the answers. As soon as there are answers, even if they are difficult to access, a motivated student will always manage to obtain them (and share them). If you want to evaluate during a practical, either distribute only the file without answers, or use a very small coefficient.

1.6.2 Detecting anomalies -

Functions have been prepared for this purpose. They allow you to test integrity at startup (`start`) or over time, by isolating the changes made, or by reporting that the hash is not as expected or has been modified. Here are some examples of implementation

```
from labquiz import QuizLab
from labquiz.putils import readData, getAllStudentsAnswers, correctAll
# # 1 - Reading data
URL = "https://URL_UTILISÉE_POUR_RECUEILLIR_LES_RÉSULTATS"
SECRET = "SECRET_PASSWORD_SPECIFIED_IN_THE_SHEET"
```



```

data, data_filt = readData(URL, SECRET)
## 2 - Students
students = sorted(list(data["student"].dropna().unique()))
## 3 - imports
from labquiz.putils import check_integrity_all_std, check_start_integrity_all_s
## 4 - test
originalp = {'retries':2, 'exam_mode:True, 'test_mode:False}
starting_values = {'exam_mode:True, 'retries:2 }
print("~~~~~")
print("=>test_start_integrity")
print("~~~~~")
check_start_integrity_all_std(starting_values, data)
print("~~~~~")
print("=>test_check_integrity_all_std")
print("~~~~~")
check_integrity_all_std(originalp, students, data)

```

The outputs are as follows (truncated with the indication [...])

```

-----
=> test check_integrity
-----
Dufour Lea: Machine modification for the same student name'
[21'b1f154204eb9e7 '04b1f154204fa9e9 ']
Dufour Lea - record 13:
    - Original key 'exam_mode changed from True to False

    - Original key 'test_mode changed from False to True
Dufour Lea - record 14:
    - Original key 'exam_mode changed from True to False
    - Original key 'test_mode changed from False to True
Dufour Lea - record 15:
    - Original key 'exam_mode changed from True to False

    - Original key 'test_mode changed from False to True
Dufour Léa - record 16:
    - Original key 'exam_mode changed from True to False
    - Original key 'test_mode changed from False to True
[...]
-----
=> test start_integrity
-----
04b1f154204fa9e9 - record 42: Original key 'exam_mode changed from True to False
04b1f154204fa9e9 - record 43: Original key 'exam_mode changed from True to False
[...]
04b1f154204fa9e9 - record 53: Original 'exam_mode key changed from True to False
21b1f154204eb9e7 - record 13: Original 'exam_mode key changed from True to False

```

```

04b1f154204fa9e9 — record 27: Original 'exam_mode key changed from True to False
-----
=> test check_integrity_all_std
-----
Dufour Léa: Machine modification for the same student name'
[21'b1f154204eb9e7 '04b1f154204fa9e9 ']
Dufour Léa — record 13:
    — Original '
exam_mode' key changed from True to False
    — Original 'test_mode key changed from False to True
Dufour Léa — record 14:
    — Original 'exam_mode key changed from True to False
    — Original 'test_mode key changed from False to True
Dufour Léa — record 15:
    — Original 'exam_mode' changed from True to False
    — Original key 'test_mode changed from False to True
[...]
J Jean —Marc — record 54:
    — Original key 'exam_mode changed from True to False
    — Original key 'test_mode changed from False to True
[...]
L Bobby — record 48:
    — Original key 'exam_mode changed from True to False
    — Original key 'test_mode changed from False to True
L Bobby — record 49:
    — Original key 'exam_mode changed from True to False
    — Original key 'test_mode changed from False to True
L Bobby — record 50:
    — Original key 'exam_mode changed from True to False
    — Original key 'test_mode changed from False to True
L Bobby — record 51:
    — Original key 'exam_mode changed from True to False
    — Original key 'test_mode changed from False to True
Lamarq Linda — record 30:
    — Original 'retries key changed from 2 to 2000000
Lamarq Linda — record 31:
    — Original 'retries key changed from 2 to 2000000
Lamarq Linda — record 32:
    — Original 'retries key changed from 2 to 2000000
Lamarq Linda — record 33:
    — Original 'retries key changed from 2 to 2000000

```

"""

Demo: Detects whether the same machine has been used for multiple student names
"""

from labquiz.putils **import** check_machine

check_machine(data_filt) *#Detects whether the same machine has been used for mu*

Same machine 04b1f154204fa9e9 used by several students‘

[Morane ‘Bob ‘L Bobby’ ‘J Jean –‘Marc ‘Legrand John’ ‘Dufour ‘Lea’
Makhoul ‘Alain ‘Lamarq Linda’]

Hash check Identification of the expected hash – the **retries**, **exam_mode**,
and **test_mode** parameters must be the same as those imposed on students.

from labquiz.utils **import** get_full_object_hash

QUIZFILE = "NAME_OF_ORIGINAL_FILE"

URL = ""

quiz = QuizLab(URL,

QUIZFILE, needAuthentication=False, mandatoryInternet=False, retries=2, exam_mod
wanted_hash = get_full_object_hash(quiz, modules = ['main', 'utils'],
WATCHLIST=['exam_mode', 'test_mode', 'retries'])

And test for everyone

from labquiz.putils **import** check_hash_integrity

check_hash_integrity(data, 'full', wanted_hash=wanted_hash)

Which provides

Dufour Léa, machine id 04b1f154204fa9e9:

The source or parameters have been modified or monkey patched

index 17 hash: ddd8b0e5a3b35c41bb8db16cda874ff52af40a9ffdf56d7510abed65a9dec69f

Dufour Léa, machine id 21b1 f154204eb9e7:

The source or parameters have been modified or monkey patched

hash found:

fb85b7bceb28ecce4a047d1fb94428c60789505ebd89e22a5154628a125fd2e8

ddd8b0e5a3b35c4 1bb8db16cda874ff52af40a9ffdf56d7510abed65a9dec69f

index [13]

J Jean –Marc, machine id 04b1f154204fa9e9:

The source or parameters have been modified or monkey patched

index 54 hash: d5aa069407214f52b187ba479047d36fead12bc5b541ba3bcece6bc9f328490

L Bobby, machine id 04b1f154204fa9e9:

The source or parameters have been modified or monkey patched

hash found:

```

880414219b2eadf47f234227c28927fc7d8a6f1a911bf40fa023f58d1a6cd83d
index [43]
7de9df303f6cbbd183bcbc3745f4c2ed87d4e9a38afc79f8a47ea47981de43a5
index [45]
d5aa069407214f52b187ba479047d36fead12bc5b541ba3bcece6bc9f328490
index [50]
Lamarq Linda , machine id 04b1f154204fa9e9 :

```

```

The source or parameters have been modified or monkey patched
hash found:
fb85b7bceb28ecce4a047d1fb94428c60789505ebd89e22a5154628a125fd2e8
4105f50a65562da899d9d9062a1853f8623c6b14b7f0a663cfc5244afd0b40
index [30]
Morane Bob, machine id 04b1f154204fa9e9 :

```

```

The source or parameters have been modified or monkey patched
index 36 hash: b78c45342e260e62883a229b0b48364956e78b039ddbd0d64b6 222d761ae8e4

```

1.6.3 Client-side security

As mentioned above, if you want to grade students or conduct an exam under proper conditions, we recommend using a question file WITHOUT answers. Since the Python source code is accessible, a student or group of students could study it, reverse engineer it, and tamper with the data sent to the server. This is reasonably complex and difficult, but it cannot be ruled out.

Therefore, “client-side” security should be added with an unpredictable intervention. This could be a supervisor walking among the rows, or the execution on demand (by the teacher or students) of a specific hidden code protected by a password that is only revealed at the time of execution. This has been implemented and should significantly reduce the temptation to cheat. The idea is to prepare a code, which will be encrypted with a password, executed on demand, and which will perform a number of integrity checks on the student’s configuration. The teacher prepares their encrypted archive as follows:

```

# example_create_secure_tar.py
# zutils.py must be present in the local directory
from labquizdev.putils import create_secure_tar
# Install python_minifier if necessary
import importlib.util
import sys
import subprocess
module_name = "python_minifier"

spec = importlib.util.find_spec(module_name)
if spec is None:
    print(f"{module_name} is not installed. Installing ...")
    subprocess.check_call([sys.executable, "-m", "pip", "install", module_name])

```

```

import python_minifier
try:
    with open('zutils.py', 'r') as f:
        minified = python_minifier.minify(f.read(), remove_literal_statement=True,
                                           rename_globals=True, preserve_globals="do_the_check")
    with open('quiz_data/zutils.py', 'w') as f:
        f.write(minified)
except:
    pass

global_hash, src_hash = create_secure_tar(
    source_dir="quiz_data",
    output_file="quiz.tar.enc",
    password_open="PASSWORD_TO_ENCRYPT_THE_ARCHIVE",

password_seal="SECOND_PASSWORD_TO_WATERMARK_SOURCES"
)

```

```

✓ Archive created: quiz.tar.enc
📁 Global archive integrity hash (to keep):
0dc72c7c4967e834e026e5c83b6a02edc1fad1956830c338350beb2e3f783f6e
📁 Global hash of source and data files (to be kept)
3abca1bb707c6b101ce8ea9427b4354a5a9005ffdfb4f30e8cc055b1f459ddc6

```

Figure 19: Execution result

Inside this archive, the `zutils` module is completely autonomous and contains a customizable `do_the_check` function that is launched automatically and performs checks that are transmitted to the server. In the version provided to teachers (but customizable, `extras` directory), a *daemon* is launched that performs the check periodically. It contains the integrity code of the `zutils.py` sources, watermarked, under the `session_hash` key. On the student side, simply call

```
quiz.check_quiz()
```

on each machine, using the `password_open` password.

```

quiz.check_quiz()
Password: *****
[Unlock]
✓ Verification performed, data sent, daemon launched
✓ Deleting /var/folders/y4/_fmvp6f90fz8qzbx1fl6pwr0000gn/T/tmpce1djndm

```

Figure 20: Execution result

1.7 Manual preparation of question file

1.7.1 Prepare and encode the question file (Manual preparation) -

The question file is a text file, so it can be prepared and maintained using a simple text editor, following the structure detailed in Section 1.4.1. Once this file has been prepared, you may want to generate

enc a base64-encoded version (to prevent it from being too easily viewed)

crypt an encrypted version with a hidden key

qo a “questions only” version without tips and answers In particular,

- For an exam, you will distribute the “questions only” file, encrypted or not (you can also use it outside of an exam, but in this case the students will have no tips, answers, or scores)
- For a test mode where you plan to provide *feedback* to students, use the version with answers, in encoded or encrypted format, adjust **retries** and set **test_mode=True** in the parameters passed to the initialization. A function allows you to perform these different operations:

```
prepare_files(input_file , output_file , mode="crypt" , pwd=''):
    """
```

```
Prepare_YAML_files_for_quizzes .
```

```
Outputs_two_files , with the basename given in 'output_file' .
```

```
The second file is questions only and is the input stripped
from responses and tips . With the 'mode="crypt" ' , the input and stripped
```

```
versions are encrypted ; with 'mode="enc" ' , both files are base64 encoded ;
```

```
finally , with 'mode=yml' , files are not encoded nor encrypted .
```

```
Parameters
```

```
~~~~~
```

```
input_file : str
```

```
Path to the input YAML file .
```

```
output_file : str
```

```
Path to the output YAML file .
```

```
mode : str , optional
```

```
Mode to prepare the file . Choose from "crypt" , "enc" , or "yaml" .
```

```
pwd : str , optional
```

```
Password for file encryption .
```

It can be used, for example, as follows

```

from labquiz.putils import prepare_files
prepare_files("quizzes_basic_filtering_test.yaml", "qbf.yml", mode="crypt", pwd=
    which produces
– Creating qbf_crypt.txt
– Creating qbf_qo_crypt.txt
File encrypted with pwd. Ensure to use the ‘mandatoryInternet=True‘ option in q

```

You will copy the password, if any, into cell A2 of the Config sheet of the Google Sheet, see Section [1.10.1](#), if you are using a GS to collect the results.

- For an exam, you will distribute the “questions only” file, encrypted or not (you can also use it outside of an exam, but in this case, students will not have tips, answers, or scores).
- For a test mode where you plan to provide * feedback* to students, use the version with answers, in encoded or encrypted format, adjust **retries** and set **test_mode=True** in the parameters passed to the initialization.

1.8 quiz_editor

1.8.1 quiz_editor - Prepare and encode the question file -

Manual preparation is possible for small databases, but a specific editor has also been created to allow editing and various operations (extraction, encoding, encryption, conversion to other formats).

This editor is available here: <https://jfb-quizeditor.streamlit.app/>

Installation In addition to cloud deployment, the application can also be installed locally using from pypi by

```
pip install quiz --editor
```

or from git

```
pip install git+https://github.com/jfbercher/labquiz.git#subdirectory=quiz_editor
```

An entry point is provided, allowing it to be used directly from the command line by simply invoking

```
quiz_editor [FILENAME.yaml]
```

Features It allows you to edit the files whose structure has been described in Section 1.4.1 and has some additional features:

[resume]

1. preview with large markdown support (formatting, lists, images, tables, maths, ...), also supported in exports
2. ability to define categories and tags, and to select questions by filtering on categories and tags,
3. define variables generation for templates,
4. extraction and saving of some of the questions,
5. preparation of an encoded or encrypted version (as described above),
6. conversion to **AMC- $\text{\LaTeX}$** format, so that questions can be reused in a paper-based AMC multiple-choice quiz, (Categories are used to define the `\element` type). There are two options available for handling “template” questions:
 - a “*static*” export, where values are generated once for the variables (based on the generation formula, e.g., `rng.integers(1,10)`) and reused for all copies. An example of output [here](#)

- a “*dynamic*” export using `pythonTeX` (a LaTeX package that must be installed), which allows different values to be generated for each copy. Thus each students’ sheets are different. This requires a small additional configuration in AMC (see footnote²). An example of output [here](#)

[resume]

1. *conversion to HTML format.* Two options available:

- **Dynamic HTML.** A “*dynamic*” export with integrated answers and correction so that a “self-assessment” web page can be easily created. This has support for templates questions, where values are dynamically regenerated³ at startup and after each reset – an example of the export result [here](#)
- **HTML exam,** without answers, and with real-time submission of results to a Google Sheet, with subsequent correction, as described in or using the dashboard as described below Section 1.9.1 – example [here](#). Support for dynamic generation of variables (each student session is different) is in progress. The export is static for now.

Commented examples The following animated GIF examples show:

- a **quick tour of the editing interface** for a multiple-choice question, demonstrating how to edit questions, options, hints, feedback, and answers; Markdown preview, category and tag definition, question selection, the export menu, etc.
- **three examples** using templates; the first two in numerical format, and the last one as an MCQ. These examples demonstrate the ability to use figures⁴ and tables in questions, to use template parameters with automatically generated values in the question stem and propositions (with optional formatting of numerical values⁵), as well as to calculate the expected solutions (Boolean or numerical)⁶.

²Use the `prePythonTex4AMC` file and add 3 lines at the beginning of the LaTeX file. A template and examples are provided in the [extras/pythontex_template](#). Some documentation (in french) is available [here](#), [here](#) or [here](#).

³Values are generated by javascript interpreting the initial python formula, with support for standard maths ops (int, float, round, min, max, abs, etc). Of course, python idioms or packages are obviously not supported in JS.

⁴For images, pandoc syntax with size attributes is supported `{width=x% height=y%}`. By default, images are inserted inline. To obtain a “displayed” image, put it on its own paragraph (blank line before and after) and use a non empty alt text (**required**) that will be used for caption. The LaTeX export will display the image in a center environment.

⁵E.g., `{a+b:3f}` computes the sum of `a` et `b` and displays the sum with 3 decimals.

⁶Note that expected values must be computed explicitly, e.g. use `round(a+b,3)` instead of `{a+b:3f}` which shall be used only for display.

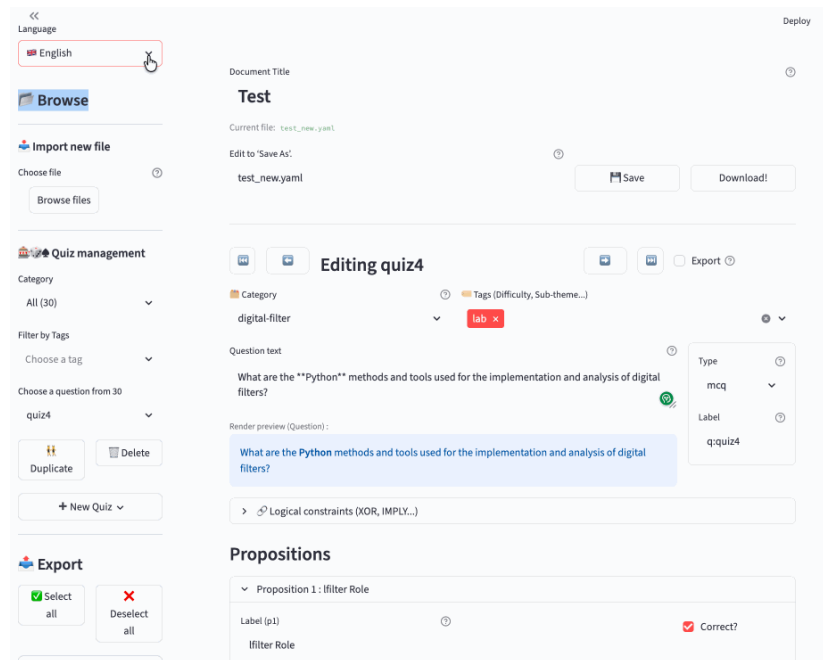


Figure 21: `quiz_editor` – Quick tour of the interface. editing a question, with category, tags, choice of question type (multiple choice, numeric, etc.), editing a proposition – correct or incorrect, hint (tip), answer (displayed during correction), bonus, penalty, etc; exports.

Category: test Tags (Difficulty, Sub-theme...): Choose or enter a new tag

> Template variables

Question text

Consider the following "confusion matrix":

```
True/Predict | 0 | 1 |
---|---|---|
0 | {TN}| {FP}|
1 | {FN}| {TP}|
```

Type: numeric-tem... Label: q:quiz79

Render preview (Question):

Consider the following confusion matrix:

True/Predict	0	1
0	48	5
1	1	43

> Logical constraints (XOR, IMPLY...)

Propositions

Figure 22: `quiz_editor` – example of numeric-template usage for a confusion matrix, with potentially external variables, including an editor preview of the table based on automatically generated variables.

Category: test Tags (Difficulty, Sub-theme...): Choose or enter a new tag

> Template variables

Question text

Consider a scatter plot such as the one shown in the [figure](#)

!{Scatter plot}(./Figure_1.png){width=38%}

We perform a linear regression "reg", which yields

```
- `reg.intercept_` = {w0:.3f} and
- `reg.coef_` = {abs(w1):.3f}.
```

For a new observation $x = \{x:.3f\}$, what would be the value predicted by the model?

Type: numeric-tem... Label: q:quiz81

Render preview (Question):

Consider a scatter plot such as the one shown in the [figure](#)



Figure 23: `quiz_editor` – example of numeric-template usage for a linear regression, with potentially external variables, including an editor preview of a scatter plot, and parameters and solution computed using the automatically generated variables.

Category

test

Tags (Difficulty, Sub-theme...)

Choose or enter a new tag

Template variables

Question text

Consider the simple regression model

$$y = \alpha x + \beta$$

with $\alpha = \text{round}(a,3)$ and $\beta = \text{round}(b,3)$

Type

mcq-template

Label

q:quiz82

Render preview (Question):

Consider the simple regression model

$$y = \alpha x + \beta$$

with $\alpha = 0.896$ and $\beta = -1.678$

Logical constraints (XOR, IMPLY...)

Propositions

Proposition 1: positive_slope

Figure 24: `quiz_editor` – example of MCQ-template usage for a linear regression, with potentially external variables, including an editor preview of a scatter plot, and parameters and solution computed using the automatically generated variables.

Keep in mind that the original idea behind specifying parameters for questions—the “template-questions”—is to use parameters obtained during experiments in a Jupyter notebook. These parameters can be provided as context for the question. For practice multiple-choice quizzes or exams, the ability to randomly generate new values allows for different test questions and different exam copies to be created.

Below, you will find different versions of a question compiled from the `PythonTeX` export, corresponding to the confusion matrix shown in editor Figure 22

An HTML practice test using the questions above can be viewed [here](#). An animated-gif shows generation and answers in the dynamic HTML output, corresponding to the confusion matrix shown in editor Figure 22

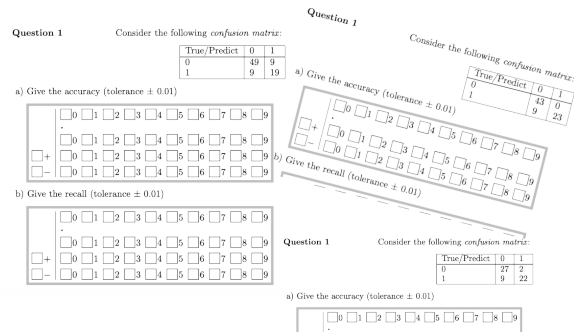


Figure 25: quiz_editor – AMC- $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ (python $\text{T}_{\text{E}}\text{X}$) export example

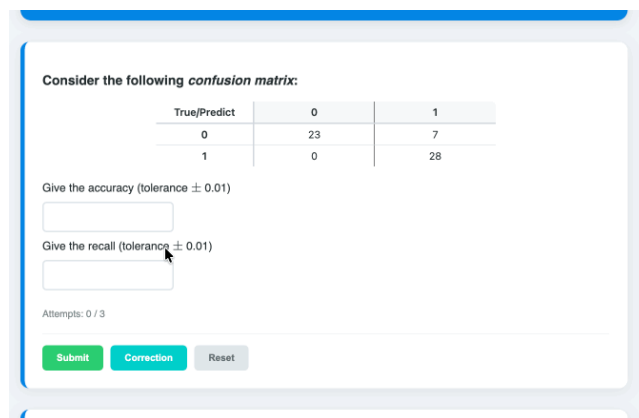


Figure 26: quiz_editor – dynamic HTML example

1.9 quiz_dash

1.9.1 quiz_dash - Monitor in real time and correct with quiz_dash -

As described above, you can load the results table at the terminal level and perform all the tests and statistics you want. However, it is easier to use a small graphical utility to do this. This monitoring and analysis dashboard is available here:

<https://jfb-quizedash.streamlit.app/>

In addition to cloud deployment, the application can also be installed locally using from pypi by

```
pip install quiz -dash
```

or from git

```
pip install git+https://github.com/jfbercher/labquiz.git#subdirectory=quiz_dash
```

An entry point is provided, allowing it to be used directly from the command line by simply invoking **quiz_dash**.

Based on the Google Sheet URL specification, the associated read password, and the YAML file containing the answers, this dashboard allows you to:

- *track over time*, with an adjustable refresh rate, the submissions made by each participant, with the labels of the relevant quizzes,
- *check integrity*, i.e., verify that the parameters (number of attempts allowed, mode, etc.) have not been modified, verify the hash of the sources, the object in memory, and its dependencies,
- *view*, over time, the progress of each participant (filterable) and of the entire group,
- *correct* and retrieve the results table,
- download pdf reports of full corrections for all participants,
- with the possibility of adjusting the weight matrix (for multiple-choice questions) and the scoring scale per question;
- analyse *participation*, with the quantification of early or late attempts as well as retakes after the main session.

Some screenshots of actual monitoring:

Since version 1.1, **quiz_dash**

- include a participation analysis sub-dashboard,
- has the ability to detect several sessions of work and use them in figures (eg. student timelines).

Figure 27: `quiz_dash` – Entering monitoring/correction configuration parameters

Figure 28: quiz_dash – Integrity monitoring

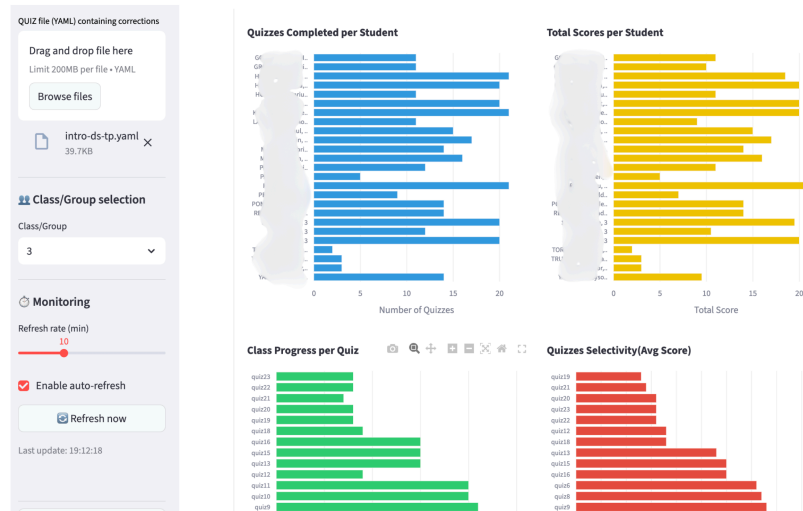


Figure 29: `quiz_dash` – Monitoring of quizzes taken by students and the group. Automatic refresh possible and adjustable (student names have been hidden)

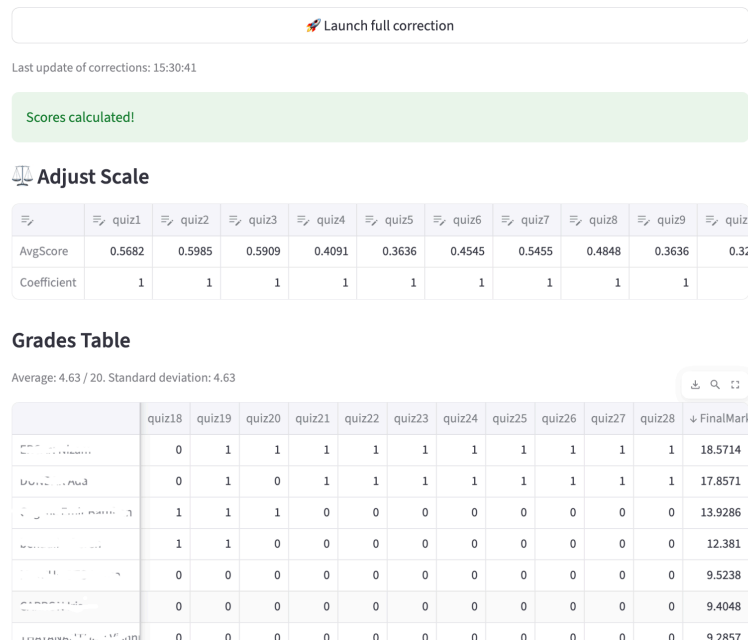


Figure 30: `quiz_dash` – Automated correction, with the option to adjust the scoring system (automatic recalculation); (student names have been hidden). Of course, the results table can be downloaded.

Select a student

> Student Timeline

Individual Report

Download PDF report Download HTML report ☒ Full correction in Report

Correction for [REDACTED]

Final mark: 71.43 / 100 -- Class mean: 54.14 & Standard deviation: 24.57

Scale:

	quiz1	quiz2	quiz3	quiz4	quiz5	quiz6	quiz7	quiz8	quiz9	quiz10	quiz11	quiz12
AvgScore	0.80	0.88	0.92	0.88	0.78	0.76	0.78	0.78	0.80	0.76	0.80	0.36
Coefficient	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00

	quiz13	quiz15	quiz16	quiz18	quiz19	quiz20	quiz21	quiz22	quiz23
AvgScore	0.56	0.60	0.60	0.36	0.28	0.32	0.28	0.32	0.32
Coefficient	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00

quiz1 - Quelles sont les dimensions et les caractéristiques structurelles de la table g1800s après son chargement initial ?

Answered at: 2026-03-11 15:30:20

☒ Elle contient 197 lignes (entrées) et 101 colonnes. **Correct** 1.0

☐ Elle contient 101 lignes et 197 colonnes. **Correct** 0.0

Figure 31: quiz_dash – Individual correction can be generated for a student.

Connection

Google Sheet URL
https://script.google.com/macros/s/

Secret Key

QUIZ file (YAML) containing corrections
Drag and drop file here
Limit 200MB per file - YAML
Browse files

Class/Group selection
Class/Group
All

Monitoring
Refresh rate (min)
10
Enable auto-refresh

Live monitoring & Correction

Integrity Live Monitoring **Correction & Grades**

Correction, Grades & Reports

Correction & Grades **Students reports**

Individual analysis

Select a student
DRIDI Rayane

> Student Timeline

> Individual Report

All Class Reports

Generate zip of all PDF reports Generate zip of all HTML reports ☒ Full correction in Report

Generating zip...

Download zip of all HTML reports

Figure 32: quiz_dash – Individual reports can be downloaded so that you can distribute them to the students.

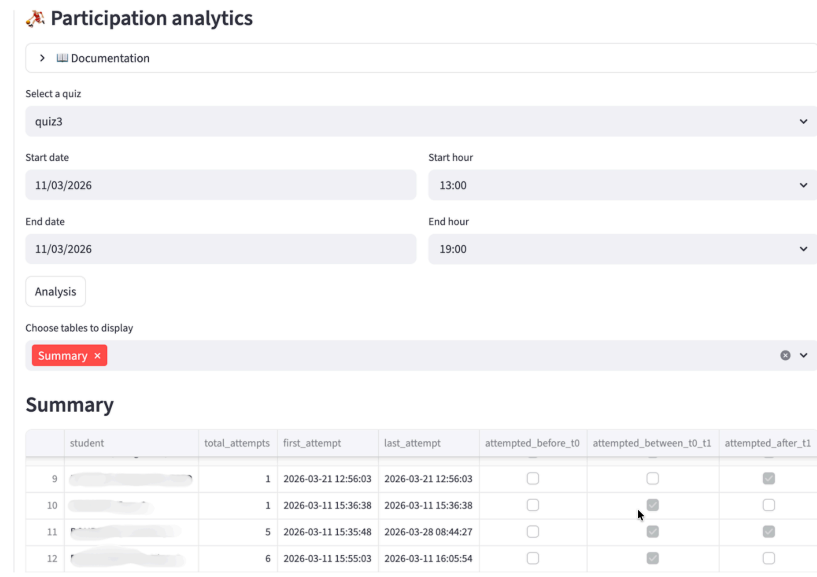


Figure 33: `quiz_dash` – Participation analytics: detects and quantify early attempts, late attempts, retakes after the main session, etc.

Since version 0.9.6, `quiz_dash`

- allows to analyze a timeline of responses and scores over time
- allows to generate detailed grading reports for all students and download the reports in HTML or PDF format for either individuals or the entire class

Since version 0.9, `quiz_dash`

- allows to take into account different classes or groups of students and display the overall results or results by group,
- has been made **persistent** with data saved in the browser's `localStorage`, allowing you to resume work in the event of a session (network) interruption, reloading, or closing the browser.

1.10 Google Sheet preparation

1.10.1 Create your Google Sheet to collect results -

To collect the data, you can simply use a Google Sheet. In a later version, a small Flask server could be used, but it would need to be deployed somewhere. For now, a Google Sheet version is functional and easy to implement.

Hint

Hint: The simplest of the simplest **The simplest of the simplest** is to use a copy of the template that has been prepared for this purpose: To do this, click on the [following link](#)

- then go to the File menu/Create a copy (rename the file),
- and deploy via: Extensions/Apps Script menu and click deploy in the top right corner; choose **Web Application**, share with everyone. Copy and save the link `https://script.google.com/macros/.../exec`, this is the URL you will use later (to share with students and use to read collected data). You can find this link by going to “Manage your deployments.”

Save and you’re done! You can browse the following if you want to create the Google Sheet yourself or understand the meaning of the parameters in the Config sheet. You need to adjust the Pwd values in the Config sheet and the SECRET value in the AppScript code (Extensions/Deploy, 1st line). See points 4, 5, and 6 below.

The simplest of the simplest...

Manual preparation

- 1 - Create a Google Sheet using your account
- 2 - On the first line of the first sheet, insert the following data, which will serve as the header for your table:

```
timestamp send_timestamp notebook_id student quiz_title  
event_type parameters answers score
```

- 3 - Create a new sheet by pressing + and rename it Config (by CTRL + clicking on the corresponding tab).

	A	B	C	D	E	F	G	H	I
1	timestamp	send_timestamp	notebook_id	student	quiz_title	event_type	parameters	answers	score

Figure 34: First row of the sheet.



Figure 35: Creation of the **Config** sheet.

- 4 - In the **Config** sheet, create the following data

Pwd Data reception
 Wrktz TRUE
 NMAX NKEEP
 2000 2500

	A	B	C
1	Pwd	Réception des données	
2	Wrktz	☒	
3			
4	NMAX	NKEEP	
5	2000	2500	
6			
7			

Figure 36: Data from the **Config** sheet.

Pwd is the key that will be used to check the correct connection with the sheet and participate in the encryption of the question file. Change the value of the key in A2 and keep it in mind.

NKEEP is the maximum number of rows kept in the sheet. Thresholding occurs as soon as the number of rows exceeds NMAX. In the **Config** sheet, select cell B2. Go to the **Data** menu and select **Data Validation**. Then, in **Data Validation Rule**, select **Checkbox**. Make sure the box is checked, otherwise you will not receive anything! *And so to stop receiving data, uncheck!* -> This can be useful because some people leave things running on their computers, and since there is a **check_alive** integrity check sent periodically, this can fill up the Google Sheet (even though we have set a maximum limit).

- 5 - In the **Extensions** menu, click on **Apps Script**. In the tab that opens, name your project, then in the code page, after deleting what is there, paste the code attached in the code_gs.txt file (**extras** folder) and replace the value of **SECRET** on the first line. If your first sheet is not called **Feuille 1**, rename it or change the constant **SHEET1** on the third line of the code.
- 6 - At the top right, click on **Deploy** then **New deployment**, choose the **Web application** type, share with everyone, copy the link <https://script.google.com/...>

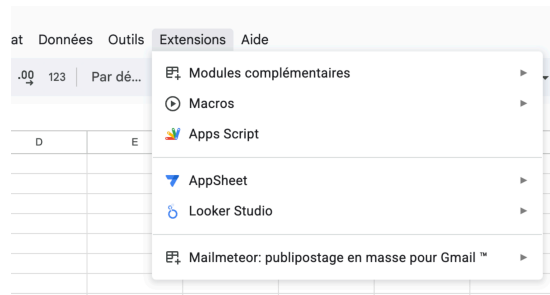


Figure 37: Create the extension to enter the Google Script code.

This is the URL you will use later (to share with students and to read the collected data). You can find this address by going to “Manage your deployments.” Save and you’re done!

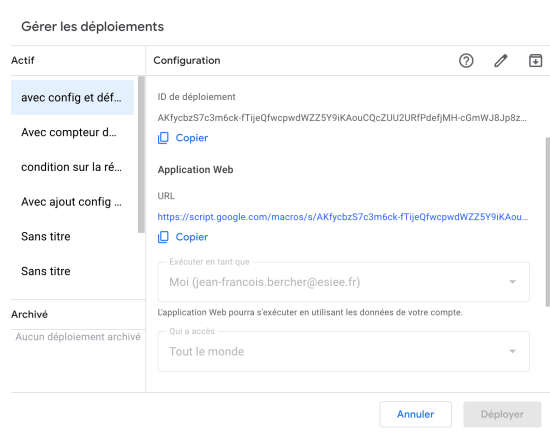


Figure 38: Deploy!

1.11 Deploy to Jupyterlite

Jupyterlite is a Python/ Jupyterlab distribution that runs entirely in the browser. The various Python packages have been ported to WebAssembly (WASM), a binary format that is executable in browsers. For optimized libraries (numpy, pandas, etc.), performance is equivalent to native performance. For pure Python (loops, etc.), execution is 3 to 10 times slower. What this means, however, is that you can prepare Python programs and Jupyter notebooks that are fully executable in a browser, with zero installation! And it works independently of the system, even on tablets, phones, etc. For example, you can find here: <https://perso.esiee.fr/~bercherj/JliteNotes/lab/> an example of a practical assignment distributed in this way, incorporating labquiz.

1.11.1 Deploy to Jupyterlite -

Preparation and deployment under Jupyterlite are not exactly immediate (there are documents to follow and adaptations to make), so I suggest a shortcut, which is as follows:

1. Download the archive of a deployed version, available here

<https://drive.google.com/file/d/14BnzVmPO6I8uOMEmNC6BTMsjXL3RLrQL/view?usp=sharing>
and unzip it on your disk,

2. Place the files you want to make available in `_output/files`
3. Run the program `update_jupyterlite_contents.py` available in the `extras` directory by typing `python update_jupyterlite_contents.py`. Install dependencies if necessary.

This will update the lists and characteristics of the files in the distribution

1. **init**) Upload *all* of the contents of `_output` to your web account, typically (or for instance...) to `login/public_html`, in a directory such as `MyJlite`, and you will obtain a distribution that is accessible and executable at <https://www.esiee.fr/~login/MyJlite>. If you don't know how to do this, ask your nearest local IT guru.
2. **update**) If you only need to perform an update, simply copy the contents of `_output/files` and `_output/api` to `login/public_html/MyJlite/files` and `login/public_html/MyJlite/api`. For subsequent updates, add or modify files, you only need to perform steps 2, 3, and 5.maj); this will save you from having to upload the 70 MB in step 4.init).

1.12 Google Authentication in QuizLab (Notebook / Web)

1.12.1 Google Authentication in QuizLab (Notebook / Web)

For instructors whose institution uses Google Workspace, it is possible, since version 1.0.0, to use Google authentication instead of the standard authentication enabled with `needAuthentication=True`. In that case, the parameter `googleAuthentication=True` must also be specified, in addition to the standard `mandatoryInternet=True`.

This short guide explains how to configure Google authentication and how to use it in QuizLab.

Note: The quiz file is specifically encrypted to enforce authentication when `googleAuthentication=True` is enabled. If students modify this parameter, they will not be able to decrypt the quiz file.

Enable authentication in QuizLab In your code, use something like

```
quiz = QuizLab(  
    URL,  
    QUIZFILE,  
    needAuthentication=True,  
    mandatoryInternet=True,  
    googleAuthentication=True,  
    retries=100,  
    groups=['A', 'B', 'C']  
)
```

```
if quiz.googleAuthentication:  
    await quiz.googleAuthenticate()
```

The following parameters are **mandatory** for authentication:

- `needAuthentication=True`
- `mandatoryInternet=True`
- `googleAuthentication=True`

For instructors: Google Cloud configuration (once) Go to: <https://console.cloud.google.com/>
(Create a project if needed)
In Google Cloud:

a. OAuth consent screen

- Menu: **APIs & Services** → *OAuthconsent screen*
 - User type: Internal
 - Fill in app name and support email
-

b. Create OAuth credentials Menu: **APIs & Services** → *Credentials* → *CreateCredentials* → *OAuthclientID*

Case 1 — Notebook / Local Python

If you intend your students to use a local desktop version of jupyter notebook or jupyter lab

- Application type: Desktop app
- Download the JSON file
- Rename it:

`client_desktop.json`

- Place it in the same directory as the notebook
-

Case 2 — Web application (e.g. JupyterLite)

- Application type: Web application
- Add an Authorized redirect URI:
 - e.g. `http://localhost:8000/` or your site URL e.g. `https://www.my_university.edu/~mypage`
- Download the JSON file
- Rename it:

`client_web.json`

- Place it next to the notebook in the web app
-

IT support (if required) Contact your IT administrator if:

- you cannot create a project
 - “Internal” option is unavailable
 - OAuth access is restricted
-

Result

- Users authenticate with their institutional Google account
 - Access is restricted to the organization
-

1.13 Demonstration and exercises

Explore the demo notebook `labQuizDemo.ipynb` available in the `extras` directory and experiment. You can also use the live [binder version](#) or even the `jupyterlite` version available [here](#)

If you want to go further, and if you’ve made it this far, you’re probably motivated, so here are a few ideas.

- **Exercise:** Create a Google Sheet named `MyFirstGSQuiz` by following the instructions in Section [1.10.1](#). Change the read password in the first line of the script accessible via Extensions/Apps Script, then deploy. Note the deployment address and password.
- **Exercise:** Load or create a question database in `quiz_editor`. Add a few questions. Save the new file. Export an HTML version and an AMC version. Upload the HTML to your account and test (or test locally).
- **Exercise:** Export an encrypted version of your questions (note the password and update it in the Google Sheet, Config page) .
- **Exercise:** Create a Jupyter notebook, import `labquiz`, instantiate a quiz, – see Section [1.3.2](#), integrate a few questions and test.
- **Exercise:** Instantiate a quiz linked to the URL of your Google Sheet, integrate a few questions into your notebook, and check that the answers appear in the Google Sheet.
- **Exercise:** Use `quiz_dash` to read the results. Change the settings in your notebook (change `mode_examen` from `True` to `False`, modify `retries`), and see if this triggers an alert. Run the correction on your few tests.
- **Exercise:** Create a `jupyterlite` version as explained in Section [1.11.1](#), add your files, upload it to your info account in `public_html`, and check by connecting to the corresponding web address.